Builder Playbook

One project, start to finish, in commands. The moves, the exact prompts, and the decision points — no implementation code.

Pairs with: the AI OS Team Guide **Sample:** a small internal web app, greenfield

Do §1 literally. Read §2 if you want the why.

No new language to learn. Two scripts, your coding agent, `git`, and typing plain requests to the desktop app. That's the whole toolset. If you read nothing else, read §1 and copy the lines.

Two surfaces, and you're always on exactly one of them. Know which:

dark block = a **terminal command** you run

dashed block = a **message you type to the desktop app**

Harness-neutral. Examples show **opencode** (the team's default) and **AGENTS.md** (the portable instruction file). On Antigravity, same `AGENTS.md`. On Claude Code, launch `claude` and symlink `CLAUDE.md → AGENTS.md`. Full swap table: the Team Guide §3.

§1 Quickstart — copy the lines

The whole loop, minimal words. Sample project: `acme-portal`. Swap the name and follow along.

```
#### ONE-TIME (per machine) ##############################
agent → "run the dev-orchestrator pre-flight check"   # Node, Git, SSH, agent — all

#### NEW PROJECT #########################################
desktop → "scaffold acme-portal as a web-app via dev-orchestrator"   # agent builds
ln -s CLAUDE.md AGENTS.md                  # one instruction file for every harness
desktop → "interview me, then fill knowledge/BRD.md and knowledge/SCHEMA.md"
# add Notion + Filesystem to your harness's MCP config ; paste the record protocol i

#### PER FEATURE — repeat until done ####################
desktop → "read playbook/features + PROGRESS.md, give me the coding-agent
           prompt for the next feature. don't write code, write the prompt."
cd acme-portal && opencode               # paste that prompt, let it build  (or: ant
  # run the project's test command — e.g. bun test / npm test
desktop → "read the changes in ~/Developer/acme-portal, verify them against
           the feature spec, flag any weakened test, then give me the next prompt."
git rev-parse HEAD                         # write this down = your revert target
git commit -am "feat: ..." && git push  # human ships. the agent never pushes.
# ↺ back to the top of PER FEATURE
```

That's it

Everything below is the same handful of lines, explained. If the quickstart already makes sense, stop reading and start building.

§2 The run, explained

Sample: **Acme Portal**, a small internal status dashboard, greenfield. The spine underneath every step is **Brain before Body** — knowledge first, source last, because an agent that codes before the spec is real just produces confident nonsense faster.

One-time, per project

0 Set the context

Desktop app

Open the scoped Project for this surface; confirm its instruction doc is pinned. Drop in what you're starting from — an idea, a whitepaper, a PDF. The quality of the whole build is set here. Don't jump to the terminal yet.

1 Check the machine, then scaffold

Terminal

Confirm the environment, then lay down the structure. The `dev-orchestrator` skill does both — trigger it from the desktop project or the agent; it runs the pre-flight check and creates the tree directly, no script.

```
# ask the agent to run the dev-orchestrator pre-flight check
# (references/claude-code-setup-guide.md, Part 8): Node, Git, SSH, agent, c
# then trigger the skill to scaffold — it reads references/project-archetyp
# and creates the acme-portal web-app tree directly. No script.
ln -s CLAUDE.md AGENTS.md          # scaffolder emits CLAUDE.md; this makes
```

What lands — Brain→Soul→Body, plus the files the agent reads:

```
acme-portal/ ├── knowledge/ ← Brain (first): BRD.md, SCHEMA.md — still empty
├── design/ ← Soul (second): look & feel ├── specs/ ← chunked 8–15k tokens:
architecture, data, API ├── playbook/ │ ├── features/ ← one file per
feature, worked in order │ └── PROGRESS.md ← updated every session ├── src/
← Body (last): the agent fills this ├── AGENTS.md ← the agent reads this
first (→ CLAUDE.md on Claude Code) └── (MCP config) ← .mcp.json /
opencode.json / ~/.gemini per harness
```

> **Decision point — risk gate**
>
> The scaffolder runs a risk assessment before writing specs. If it returns 🔴 High or ⛔ No-Go, stop and revise scope *now* — not after the agent has built a week of code on a bad premise.

2 Fill the Brain

Desktop app

The scaffolder interviews you — vision, platforms, stack, key flows, trade-offs. Its answers become `knowledge/BRD.md` (what it does) and `knowledge/SCHEMA.md` (how the data relates). Hold the line: **no `src/` until the Brain is real.** An agent building against an empty spec is the most common way these go sideways.

3 Wire the record

Terminal + Notion

Give the harness its memory. Add Notion to your harness's MCP config, create this project's **Project State** row, and paste the protocol into `AGENTS.md`.

```
# MCP config — same server, config file differs per harness (the Team Guide
#   Claude Code   .mcp.json          { "notion": { "type":"http", "url":"ht
#   Antigravity   ~/.gemini/…        { "notion": { "serverUrl":"https://mcp
#   opencode      opencode.json      (mcp section — see opencode docs)
# also add the Filesystem MCP here — it's what lets desktop verify from sou

# AGENTS.md — the record protocol, as rules the agent follows
START read Project State <row-id> + Canonical Facts <page-id>
END   rewrite Project State if it moved; log decisions to <db-id>
```

> **Filing reminder (the Team Guide §2)**
>
> The **company**, your **project states**, your **decisions**, and the **canonical IDs** go in Notion. The **BRD, schema, and specs** stay in the repo. Project State is a one-line row that says where this project stands — not the place for the spec.

Per feature — the build cycle

This is the loop you'll live in. A four-beat ping-pong between the desktop app (which plans and verifies) and your coding agent (which builds). You are the hand carrying prompts across.

REPEAT PER FEATURE

A Desktop writes the prompt

Desktop app

Don't hand-write prompts for the coding agent. Ask the desktop project to write them — it has the specs and the record, so it writes better ones than you will.

You → Desktop

"Read `playbook/features/FEATURE_01_auth.md` and `PROGRESS.md`. Write the exact prompt for the coding agent to implement Feature 01 — name the files to touch, the tests to add, and the acceptance check. Don't write code; write the prompt."

Copy what it gives back.

B The coding agent builds

Terminal

Open the agent, paste the prompt, let it work. Then run the project's tests.

```
cd acme-portal && opencode     # or: antigravity · claude
# paste the prompt from step A, let it implement
bun test                       # the project's test command (npm test, (
```

Context hygiene

Break at feature boundaries, not mid-feature. Keep `PROGRESS.md` current so a fresh session re-reads `AGENTS.md` + `PROGRESS.md` and knows where it is. Each harness has its own way to trim or restart a long session (Claude Code `/compact`, opencode session revert, Antigravity's manager) — use it before context gets heavy.

C Desktop verifies, and writes the next prompt

Desktop app

Back to the desktop project. Because it can read the repo directly over the Filesystem MCP, you don't paste the diff — you point it at the folder.

You → Desktop

"Read the changes in `~/Developer/acme-portal`. Verify Feature 01 against `FEATURE_01_auth.md`: did it meet the spec, were tests added, was any test weakened to pass? If it's good, give me the prompt for the next feature. If not, give me the prompt to fix it."

It reads the actual files, gives you a verdict, and hands back the next prompt — which is step A again.

> **If your desktop project can't see the repo**
>
> No Filesystem MCP? Fall back to pasting the diff: run `git diff` in the terminal, copy the output, and give it to the desktop with the same verify request.

D Human ships

Terminal

Only once the desktop says it's verified. Note the revert target first, then push. The agent never does this.

```
git rev-parse HEAD                    # write this hash down = revert ta
git commit -am "feat: auth" && git push  # human ships
```

↻ next feature → back to A

> **What just happened**
>
> Desktop wrote the prompt → you carried it to the agent → the agent built → you carried the result back → desktop verified from source and wrote the next prompt → you shipped. The record updated itself over MCP at session end. That four-beat carry is the entire day-to-day. Everything else is setup you did once.

§3 Variants & graduation

Existing repo, not greenfield

Skip stage 1's scaffold. Drop an `AGENTS.md` and your harness's MCP config into the repo, create the Project State row (stage 3), and start the cycle at step A. You inherit the code; you're wrapping the loop around it.

When you've done this enough

Once features are testable specs and you trust the loop, stop carrying prompts by hand. Declare the feature as a **spec** — goal, mandatory verifier, constraints, autonomy notch — and let the runtime decompose, dispatch, run, and verify it across whichever harness fits (it drives opencode, Claude Code, headless, and n8n alike). The verifier's verdict outranks the agent's self-report; autonomy is earned on clean runs. That's the graduation (the Team Guide §7). Don't reach for it before the four-beat carry is second nature.